

QUARTERLY AWARDS · INNOVATION

Enterprise Risk Management (ERM)

2 developers rebuilt a business-critical, undocumented app for Airbus DevX in one PI, with an AI method any team can reuse

[Team name] · [Presenter names] · [Date]

ERM is business-critical for 892 risk managers

LIFETIME UNIQUE USERS

892

Across engineering, programmes and operations

MONTHLY ACTIVE PEAK

320+

Active risk managers per reporting month

DASHBOARDS CREATED

1,600+

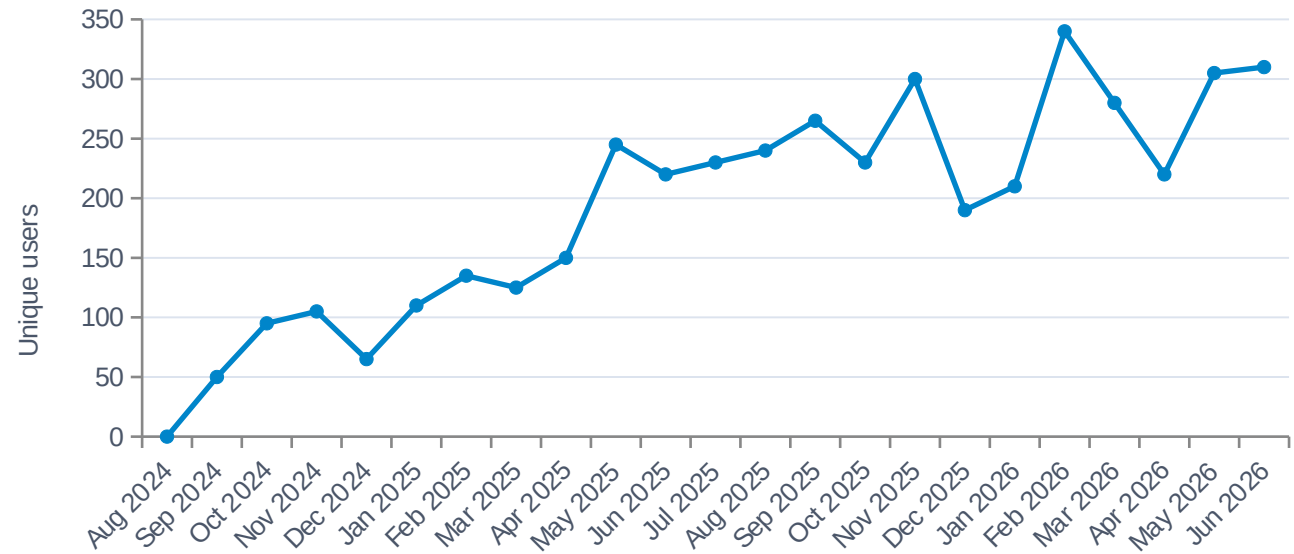
Custom dashboards created to date

UPDATED DAILY

10+

Maintained continuously by risk owners

MONTHLY UNIQUE USERS TREND



Approximate monthly values, read from the ERM usage chart

WHAT ERM DOES

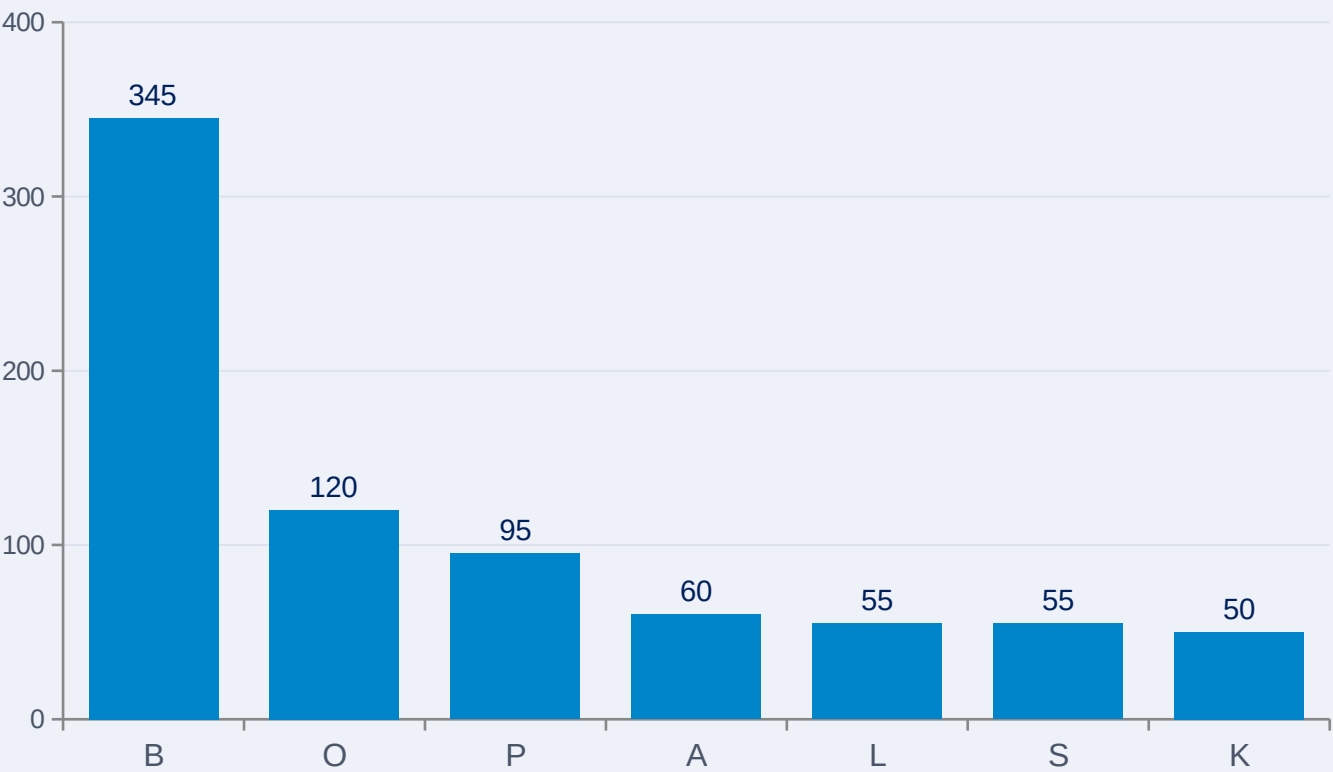
One place to manage ARM risks and opportunities: custom dashboards, live editing, trend following, governed quarterly releases and cross-division sharing.

Every risk and opportunity review in ERM depends on it working.

Used across Airbus organisations and sites

More than half of ERM users work outside its largest organisation, across 6 countries

UNIQUE USERS BY ORGANISATION (SIGLUM 1)



Plus about 12 smaller organisations (D, M, Z, Q, G, F, H, I, V, E, T...). Approximate values, read from the ERM usage chart

LOCATION PROFILE

- France**
Blagnac Wings Campus 271, Saint-Martin 43 users
- Germany**
Hamburg Finkenwerder 205 users
- United Kingdom**
Broughton 32, Filton 26 users
- Spain**
Getafe 25 users
- Canada**
Mirabel 12 users
- India**
Bangalore 7 users

But Slate had become the bottleneck

Tangled dependencies

One user action cascades through widgets, events, actions and variables

High regression risk

Changing one feature can break unrelated dashboard behaviour

Slow navigation

Variable trees are recalculated on every tab switch

No documentation

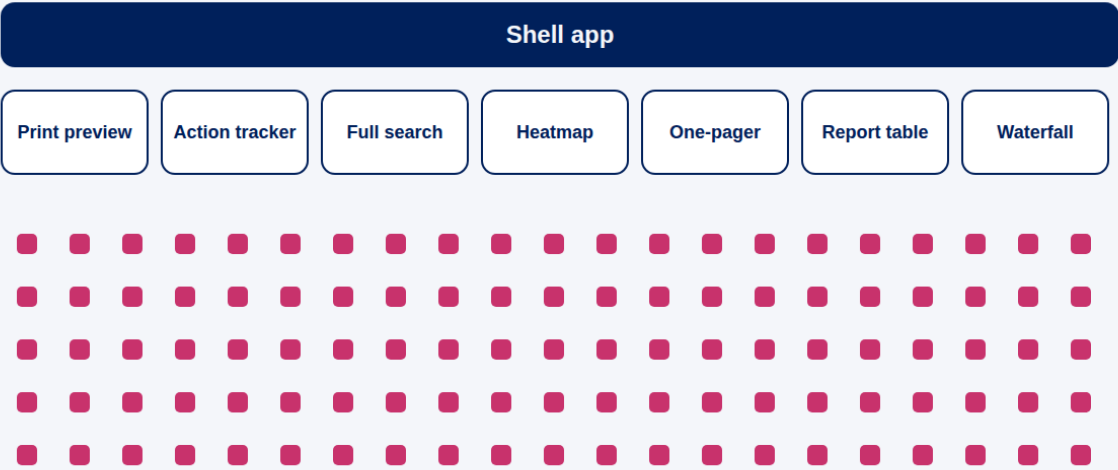
8 Slate exports, about 25 MB of configuration, no specs

Step 2: AI extracts every feature

Each module mined for widgets, events, queries and actions

105

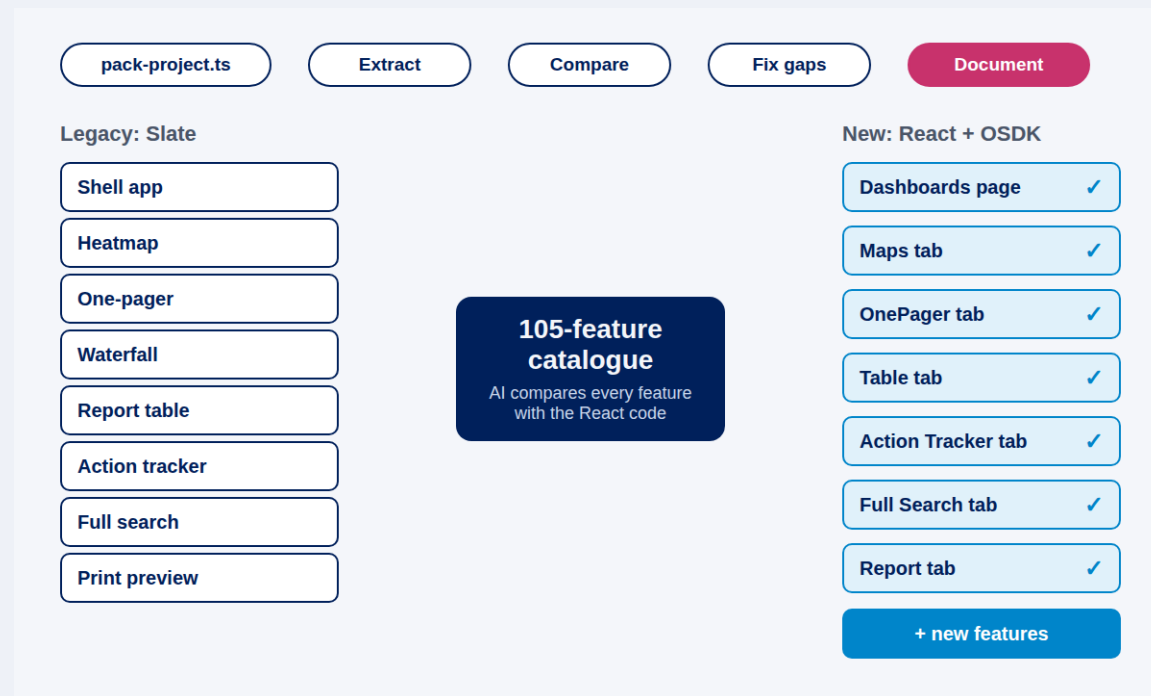
features found



THE QUESTION

How do 2 developers move 105 hidden features in one PI, without breaking risk reporting?

Our answer: AI reverse-engineers, people verify



pack-project.ts

Our script packs the whole React app, 75 files, into one dated context the AI reads in one go. The 8 Slate exports complete the picture.

USUAL MIGRATION

Discovery

Interviews and clicking through screens

Context

Code read in fragments

Parity check

Manual testing, gaps found late

Documentation

Written afterwards, if at all

OUR METHOD

Discovery

AI reads all 8 Slate exports: 105 features catalogued

Context

pack-project.ts gives the AI the whole app at once

Parity check

Every feature checked against the code: 31 gaps found

Documentation

8 architecture and lifecycle diagrams made alongside

Same features, one PI, two developers

1 PI

for the whole migration

2

developers

105

features carried over
and checked

~\$80K

**saved: about 76% less effort
than a manual migration**

4 sprints (≈ \$25K) actual vs ~17 sprints (≈ \$105K) estimated for 2 developers
Generic estimate: 105 features × 3 days + 25 days docs ≈ 340 person-days, at \$38.50/h

31

gaps found and
prioritised by AI review

7

security and data-
integrity issues caught in
review

0 → 10

documents: 8 diagrams
and 2 guides

READY FOR GO-LIVE · [planned date]

Ready now

All 105 features checked against the code;
backend sharing fix deployed

Next

Go-live to 892 users across 6 countries, then
retire the Slate app

We will measure

Users retained, page-switch time against
Slate, incidents after release

Faster to build, cheaper to run

11

new features Slate never had, shipped in the same PI

11 files

one change added app-wide notifications and auto-refresh

−97%

25 MB of Slate configuration became 0.8 MB of typed code

~\$10K

less maintenance effort every year (estimate)

ADDING A FEATURE IN SLATE

- Wire widgets, events, variables and functions across 8 exports
- Any change can break unrelated dashboards
- No documentation to know what depends on what

ADDING A FEATURE IN REACT + OSDK

- One isolated, typed component plus one OSDK call
- pack-project.ts gives AI the whole app as context
- Architecture and lifecycle diagrams show the impact

Estimates at \$38.50 per developer hour, 8 hours a day. Maintenance: rule of thumb of ~20% of build effort a year (340 person-days → 68 a year, ≈ \$21K), assumed halved by modular, typed, documented code (≈ \$10K a year saved).

What the business says

Feedback from ERM business stakeholders, September 2026

“

Thanks to a mix of regular teamwork with the business and strong IT skills, they have turned an idea into a user-friendly industrial [solution]... User interface and performance have significantly improved.

ERM Centre of Competence
Enterprise Risk Management

“

The Skywise ERM dashboard application is a tremendous addition to our ERM toolset... More and more users regularly prepare and facilitate risks & opportunities reviews with the application.

ERM stakeholder
[Role]

“

They built great new features that make navigation easier and give us a much clearer view of our R&O data. Whenever bugs or issues came up, they fixed them quickly and efficiently.

Risk and Opportunity Manager
Airbus Operations GmbH, Hamburg

Four 2026 objectives, one team effort

OUR PROJECT

EVIDENCE

2026 OBJECTIVE

AI-assisted migration of ERM

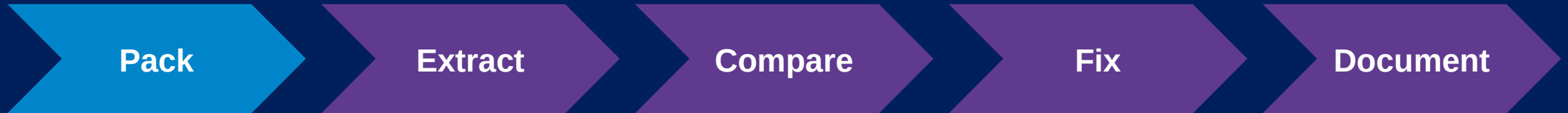
Slate to React + OSDK
2 developers · 1 PI
pack-project.ts

- AI read 8 Slate exports and the packed React code: 105 features recovered, 31 gaps found
- Delivered in 1 PI by 2 developers, about 76% less effort than a manual estimate
- All 105 features tracked for parity; ERM rebuilt on a standard, maintainable stack, ready to leave Slate
- pack-project.ts and the playbook let any team repeat the method

- **Leverage AI to solve business problems**
India TFO · Digital: democratise AI
- **Deliver on time, on cost, on quality**
Digital: optimise our digital resources
- **Ensure our operational performance**
Digital: availability of systems for operations
- **Grow digital acumen and competencies**
Digital: develop our people · India TFO

BUILT TOGETHER Business users confirmed the feature catalogue · Data and security owners kept restricted views intact · Backend and frontend fixes shipped together

A playbook any team can run next



Ready to reuse

- pack-project.ts: whole-app context in one command
- Feature comparison template
- Architecture and lifecycle diagram formats

Where next

- Functions beyond DMS, via the architecture sync
- Other Slate apps in [business areas]
- Any low-code tool with little documentation
- Any team starting an AI-assisted migration

Our ask

Endorse the playbook as a DevX pattern. We have aligned with [Architect] to propose it at the architecture sync, so functions beyond DMS can use it.

[Next: a first team outside DMS]

105 features, one PI, two developers. Ready for go-live, and for the next app.